

ANALISIS DE OPTIMIZACION Y REFACTORIZACION

Agente Autonomo Local v13

ReAct + Triple Memoria + Ollama

Hardware	Software	Estado
RTX 3060 12GB	Ollama v0.30.7 + CUDA	v13 Produccion
16GB RAM	qwen2.5:14b + llama3.1:8b	2397 lineas
Windows 10/11	Python + Streamlit	15 herramientas

Documento generado el 11 de junio de 2026

Tabla de Contenidos

1. Diagnostico del Estado Actual	3
1.1 Estructura del Archivo Actual	3
2. Problemas Criticos de Recursos	3
2.1 Consumo de Memoria por Embeddings	3
2.2 Llamadas Redundantes al LLM	4
2.3 VectorStore: Carga y Persistencia Ineficiente	4
3. Plan de Refactorizacion Arquitectonica	4
3.1 Estructura de Modulos Propuesta	4
4. Optimizaciones Detalladas por Componente	5
4.1 Optimizacion de Embeddings y VectorStore	5
4.1.1 Formato Binario para Vectores (pickle/numpy)	6
4.1.2 Cache LRU con OrderedDict	6
4.1.3 Busqueda Vectorizada con numpy	6
4.2 Optimizacion del Motor LLM	6
4.2.1 Conexion Ollama Persistente	6
4.2.2 Compresion de Contexto Inteligente	7
4.2.3 Timeout Inteligente y Retry Adaptativo	7
4.3 Optimizacion del Sistema de Herramientas	7
4.3.1 Registry Dinamico de Herramientas	7
4.3.2 Ejecucion Paralela de Herramientas Independientes	7
5. Plan de Implementacion por Fases del Roadmap	8
5.1 Fase 1: ReAct + Function Calling (Completada)	8
5.2 Fase 2: Triple Memoria + Contexto Enriquecido (80% Completada)	8
5.3 Fase 3: MCP + Metacognicion	9
5.4 Fase 4: Multimodal + Guardrails	9
5.5 Fase 5: Multi-Agente + Workflow Engine	9
6. Resumen de Impacto Estimado	10

1. Diagnostico del Estado Actual

El agente autónomo local v13 es un sistema funcional que ha evolucionado significativamente desde sus primeras versiones. Sin embargo, después de 13 iteraciones acumuladas en un único archivo de 2397 líneas, el proyecto presenta múltiples oportunidades de optimización que impactan tanto el rendimiento en producción como la mantenibilidad a largo plazo. Este análisis identifica los cuellos de botella más críticos, las redundancias de código, y las oportunidades de refactorización arquitectónica que permitirán al proyecto escalar hacia las Fases 3-5 del roadmap sin colapsar bajo su propio peso técnico.

1.1 Estructura del Archivo Actual

El archivo `app_auto_pro.py` contiene 2397 líneas que mezclan seis responsabilidades distintas sin separación de concerns: configuración global, definición de herramientas, sistema de aprendizaje, triple memoria, motor ReAct con LLM, e interfaz Streamlit. Esta estructura monolítica dificulta las pruebas unitarias, impide la reutilización de componentes, y hace que cualquier cambio en una sección pueda introducir regresiones inesperadas en otra. A continuación se detalla la distribución actual:

Seccion	Lineas	Porcentaje	Responsabilidad
Configuracion	1-141	6%	Constantes, modelos, sitios web
Herramientas	142-722	24%	15 tools + busqueda de apps
Tool Schemas	723-965	10%	Esquemas JSON + mapa de funciones
Learning System	968-1050	4%	Correcciones y conocimiento
Triple Memoria	1050-1547	21%	Embeddings, VectorStore, TripleMemory
LLM + ReAct	1548-2184	27%	Ollama, <code>_llm_generate</code> , <code>ReactAgent</code>
Streamlit UI	2185-2397	9%	Interfaz grafica completa

Tabla 1: Distribucion de lineas por seccion en `app_auto_pro.py` v13

2. Problemas Criticos de Recursos

2.1 Consumo de Memoria por Embeddings

El sistema de embeddings actual presenta tres problemas de recursos que se agravan con el uso prolongado. Primero, el cache global `_EMBED_CACHE` almacena vectores completos (768 dimensiones en `nomic-embed-text`) como listas de Python, lo cual consume aproximadamente 6KB por entrada. Con el limite actual de 200 entradas, esto representa 1.2MB en vectores, pero el overhead de las listas de Python es 3-4x mayor que los datos reales, elevando el consumo real a 4-5MB. Segundo, la eviction FIFO actual elimina la mitad del cache cuando se alcanza el limite, lo cual causa picos de llamadas a Ollama tras cada limpieza. Tercero, el VectorStore carga todos los vectores en memoria al primer acceso

y nunca los libera.

Impacto: Con 1000 entradas en VectorStore (max_entries), los vectores consumen ~25MB de RAM solo en datos de embeddings, mas el overhead de Python. En un sistema con 16GB RAM compartido con Ollama (que ya usa 8-10GB para qwen2.5:14b), esto reduce el margen disponible.

2.2 Llamadas Redundantes al LLM

Cada iteracion del bucle ReAct invoca `_llm_generate()`, que internamente construye los mensajes desde cero incluyendo todo el contexto de Triple Memoria. Esto significa que en una conversacion tipica de 8 iteraciones, el sistema realiza 8 llamadas completas al LLM donde cada una incluye: el system prompt completo, el historial de conversacion, el contexto de memoria, y las definiciones de herramientas. No existe ningun mecanismo de cacheo de respuestas para consultas identicas, ni un sistema de "context window" que evite reenviar informacion que el modelo ya procesó en iteraciones anteriores. En terminos practicos, esto significa que cada mensaje del usuario puede generar entre 8 y 16 llamadas HTTP a Ollama, con latencias de 2-5 segundos cada una.

2.3 VectorStore: Carga y Persistencia Ineficiente

El VectorStore actual guarda vectores en un archivo JSON (`vectors.json`) que se carga completamente en memoria al primer acceso. Cada operacion `add()` escribe el archivo completo al disco, y cada `search()` calcula similitud coseno contra TODOS los vectores almacenados, lo cual es $O(n)$ en el numero de entradas. Con 500 entradas de 768 dimensiones, una busqueda requiere 384,000 multiplicaciones punto, lo cual toma ~50ms en Python puro pero podria reducirse a ~5ms con numpy optimizado o a menos de 1ms con un indice ANN (Approximate Nearest Neighbor). Ademias, el metodo `_save_vectors()` escribe todo el diccionario como JSON en cada insercion, lo cual es extremadamente lento para stores grandes.

3. Plan de Refactorizacion Arquitectonica

La refactorización propuesta divide el archivo monolítico en un paquete Python modular que sigue el principio de responsabilidad única. Cada módulo encapsula una funcionalidad coherente y expone una API limpia, lo cual facilita las pruebas, el mantenimiento y la extensión del sistema. La nueva estructura tambien prepara el terreno para la Fase 3 (MCP) y la Fase 5 (Multi-Agente), que requieren componentes desacoplados que puedan ejecutarse de forma independiente.

3.1 Estructura de Modulos Propuesta

Archivo	Lineas Est.	Contenido
<code>__init__.py</code>	5	Importaciones publicas del paquete

config.py	120	Constantes, modelos, paths, sitios web
tools/	-	Paquete de herramientas
commands.py	180	ejecutar_comando, seguridad, validacion
files.py	120	leer, escribir, listar, buscar en archivos
apps.py	200	abrir_aplicacion, buscar_exe, start_menu
web.py	80	abrir_url, buscar_youtube, buscar_web
dev.py	150	generar_codigo, clonar_repo, instalar_dep
system.py	80	procesos_activos, matar_proceso
schemas.py	250	TOOL_SCHEMAS + TOOL_FUNCTIONS registry
memory/	-	Paquete de memoria
embeddings.py	80	Cache de embeddings, _get_embedding
vectorstore.py	200	VectorStore con binary format
triple.py	250	TripleMemory + session persistence
learning.py	100	LearningSystem (correcciones, conocimiento)
agent/	-	Paquete del motor de agente
llm.py	200	Ollama client, _llm_generate, fallbacks
react.py	250	ReactAgent, JSON fallback, tool execution
prompts.py	100	SYSTEM_PROMPT, JSON_TOOLS_PROMPT
ui/	-	Paquete de interfaz
streamlit_app.py	200	Interfaz Streamlit completa
main.py	15	Entry point: from ui.streamlit_app import main

Tabla 2: Estructura modular propuesta (de 1 archivo a 15+ modulos)

4. Optimizaciones Detalladas por Componente

4.1 Optimizacion de Embeddings y VectorStore

La optimización del subsistema de memoria vectorial es la de mayor impacto inmediato en rendimiento. Actualmente, cada búsqueda semántica realiza un escaneo lineal $O(n)$ contra todos los vectores almacenados, y cada inserción escribe el store completo a disco como JSON. Las tres optimizaciones

propuestas a continuacion reducen la latencia de busqueda en 10x y el consumo de memoria en 3x sin cambiar la interfaz publica.

4.1.1 Formato Binario para Vectores (pickle/numpy)

Reemplazar el almacenamiento JSON de vectores por numpy arrays serializados con pickle. Un vector de 768 float32 ocupa 3072 bytes en binario, pero 5000+ bytes en JSON (por los separadores y decimales). Con 1000 entradas, el archivo pasa de 5MB (JSON) a 3MB (pickle). Mas importante aun, cargar un numpy array desde pickle es 50x mas rapido que parsear JSON, y las operaciones de similitud coseno pueden vectorizarse con operaciones de matriz numpy en lugar de bucles Python. La implementacion requiere unicamente cambiar los metodos `_save_vectors()` y `_get_vectors()` del `VectorStore`, manteniendo la misma interfaz.

4.1.2 Cache LRU con OrderedDict

El cache de embeddings actual usa un diccionario simple con eviction FIFO que elimina la mitad de las entradas cuando se llena. Esto causa picos de miss rate tras cada limpieza. Reemplazar por `collections.OrderedDict` con eviction LRU (Least Recently Used) elimina una entrada por vez, manteniendo las mas utilizadas. Ademas, aumentar el limite de 200 a 500 entradas es seguro: 500 vectores x 6KB = 3MB, insignificante frente a los 8GB que usa Ollama. Esta optimizacion reduce las llamadas repetidas a Ollama `/api/embeddings` en un 40-60% durante conversaciones largas donde se buscan temas recurrentes.

4.1.3 Busqueda Vectorizada con numpy

La funcion `_cosine_similarity()` actual calcula la similitud entre dos vectores con un bucle Python, lo cual es extremadamente lento para busquedas contra stores grandes. La optimizacion consiste en almacenar todos los vectores como una matriz numpy (shape: `n_entries x 768`) y usar operaciones de algebra lineal vectorizadas para calcular todas las similitudes en una sola operacion. Con numpy, la busqueda en 1000 vectores pasa de ~50ms a ~3ms, un speedup de 15x. La implementacion es directa: pre-normalizar los vectores al insertarlos, y usar dot product para la busqueda (equivalente a cosine similarity cuando los vectores estan normalizados).

4.2 Optimizacion del Motor LLM

4.2.1 Conexion Ollama Persistente

El sistema actual intenta reconectar a Ollama en cada llamada a `_llm_generate()` si la conexion cacheada falla, probando multiples hosts y metodos. Esto agrega 2-3 segundos de latencia en cada reconexion. La optimizacion consiste en establecer la conexion una vez al iniciar la aplicacion y mantenerla viva con un heartbeat periodico (ping a `/api/tags` cada 30 segundos). Si la conexion se pierde, se reintenta con backoff exponencial (1s, 2s, 4s, 8s) en lugar de probar todas las combinaciones inmediatamente. Ademas, el client singleton actual (`_ollama_client`) no se recrea nunca una vez creado, lo cual es correcto para eficiencia pero problematico si Ollama se reinicia, ya que el client queda en un estado invalido sin mecanismo de recuperacion.

4.2.2 Compresion de Contexto Inteligente

Cada llamada al LLM envia el system prompt completo, todo el historial de conversacion, el contexto de memoria, y las definiciones de herramientas. Para qwen2.5:14b con contexto de 32K tokens, esto es aceptable al inicio pero se degrada rapidamente en conversaciones largas. La optimizacion propuesta implementa tres niveles de compresion: primero, truncar respuestas anteriores del asistente a un resumen de 100 caracteres en lugar de enviar el texto completo; segundo, reemplazar las 15 definiciones de TOOL_SCHEMAS por una version comprimida que omita descripciones detalladas cuando se usa JSON fallback; tercero, implementar un cache de contexto que evite recalculer el contexto de memoria si el mensaje del usuario es similar al anterior (usando similitud coseno del embedding del mensaje). Esta optimizacion puede reducir el uso de tokens en un 30-50% en conversaciones de mas de 10 intercambios.

4.2.3 Timeout Inteligente y Retry Adaptativo

El timeout actual es fijo: 180s para modelos de 14b+ y 120s para el resto. Sin embargo, el tiempo de respuesta de Ollama varia dramaticamente segun la carga del sistema y la longitud del prompt. La optimizacion implementa un timeout adaptativo basado en el historial de respuestas: se mantiene un promedio movil de los ultimos 10 tiempos de respuesta, y el timeout se configura como 2x el promedio + 30s. Ademas, se implementa retry con backoff exponencial para errores transitorios (timeout, connection reset), evitando el patron actual de probar todas las combinaciones de host/metodo/modelo en cada fallo.

4.3 Optimizacion del Sistema de Herramientas

4.3.1 Registry Dinamico de Herramientas

Actualmente, las 15 herramientas estan divididas en tres secciones del codigo (herramientas basicas, herramientas v11, y busqueda de aplicaciones) con sus esquemas JSON definidos en una lista separada y un diccionario de funciones que las mapea. Este patron hace que agregar una nueva herramienta requiera tocar tres archivos: la funcion, el esquema, y el diccionario. La refactorizacion propone un decorador `@tool` que registra automaticamente la funcion y su esquema en un registry centralizado. Esto reduce el boilerplate de agregar herramientas de 3 puntos de edicion a 1, y permite cargar herramientas dinamicamente desde plugins en la Fase 3.

4.3.2 Ejecucion Paralela de Herramientas Independientes

El bucle ReAct actual ejecuta herramientas secuencialmente, una por iteracion. Sin embargo, en muchos casos el agente podria ejecutar multiples herramientas independientes en paralelo (por ejemplo, leer un archivo Y buscar en web simultaneamente). La optimizacion implementa deteccion de independencia: si el LLM devuelve multiples tool calls en una respuesta, se ejecutan en paralelo usando `concurrent.futures.ThreadPoolExecutor`. Esto reduce la latencia total de 2 llamadas secuenciales de 5s cada una a 5s en paralelo, un speedup de 2x. Es importante notar que el soporte para multiples tool calls ya existe parcialmente en el codigo actual, pero solo se ejecuta el primer tool call de la lista.

5. Plan de Implementacion por Fases del Roadmap

A continuacion se detalla como integrar las optimizaciones y refactorizaciones propuestas con el roadmap de 5 fases del proyecto. Cada fase incluye las optimizaciones de recursos que corresponden, la refactorizacion necesaria para soportar las nuevas funcionalidades, y las dependencias entre fases. El enfoque es incremental: cada fase deja el proyecto en un estado estable y funcional, sin romper las funcionalidades existentes.

5.1 Fase 1: ReAct + Function Calling (Completada)

Aspecto	Estado	Notas
ReAct Loop	Completo	8 iteraciones max, think-act-observe
Function Calling	Completo	Nativo (qwen3) + JSON fallback (qwen2.5)
15 Herramientas	Completo	Incluye abrir_url, buscar_youtube
Multi-modelo	Completo	Auto-deteccion, fallback automatico
Seguridad	Completo	COMANDOS_PELIGROSOS + path traversal

5.2 Fase 2: Triple Memoria + Contexto Enriquecido (80% Completada)

La Fase 2 esta funcionalmente completa al 80%. Las funcionalidades implementadas incluyen: VectorStore casero con embeddings de Ollama, cache de embeddings con eviction FIFO, TripleMemory con budget de contexto (3000 chars), persistencia de sesion con TTL de 24 horas, y resumen de conversacion (simple y LLM). Sin embargo, quedan pendientes optimizaciones criticas que afectan el rendimiento en produccion y que son prerequisite para las fases posteriores.

Optimizacion Pendiente	Impacto	Tiempo	Prioridad
VectorStore: formato binario (pickle/numpy)	3x menos RAM, 50x carga mas rapida	2h	Alta
Embedding cache: LRU con OrderedDict	40-60% menos llamadas a Ollama	1h	Alta
Busqueda vectorizada con numpy	15x mas rapido en stores grandes	2h	Alta
VectorStore: append-only (no reescribir todo)	Escritura instantanea vs 100ms+	1.5h	Media
TripleMemory: compresion de contexto inteligente	30-50% menos tokens por llamada LLM	3h	Media
Qdrant integration (opcional, upgrade futuro)	Busqueda ANN sub-millisecond	4h	Baja

Tabla 3: Optimizaciones pendientes de Fase 2 con estimaciones

5.3 Fase 3: MCP + Metacognition

La Fase 3 introduce dos capacidades transformadoras. Primero, el protocolo MCP (Model Context Protocol) permite que el agente se comunique con servidores de herramientas externas, exactamente como lo hace Open Design con su comando `od mcp install`. Esto transforma el agente de un sistema cerrado con 15 herramientas hardcodeadas a una plataforma extensible donde cualquier servicio puede exponer capacidades al agente. Segundo, la metacognition permite que el agente evalúe su propia confianza en las respuestas, decida cuando necesita mas informacion, y detecte cuando esta entrando en un bucle improductivo en el ReAct loop.

La refactorizacion modular propuesta en la Seccion 3 es un prerrequisito critico para esta fase. Sin ella, implementar MCP requeriria modificaciones en al menos 8 puntos diferentes del archivo monolitico, con alto riesgo de regresiones. Con la estructura modular, MCP se implementa como un nuevo modulo `agent/mcp_client.py` que se integra al ReactAgent a traves del registry de herramientas, sin modificar el codigo existente. La metacognition se implementa como un decorador que envuelve las llamadas al LLM, evaluando la confianza de la respuesta y decidiendo si se necesita una iteracion adicional.

Componente	Descripcion	Tiempo
Refactor: monolito a modulos	Separar en paquetes tools/, memory/, agent/, ui/	4-6h
MCP Client	Conexion a servidores MCP externos, discovery de tools	3-4h
MCP Tool Registry	Fusionar tools locales + MCP en un registry unificado	2h
Metacognition: confianza	Auto-evaluacion de certeza en respuestas	2h
Metacognition: loop detection	Detectar iteraciones ReAct improductivas	1.5h
Testing integrado	Tests unitarios por modulo + tests de integracion	2h

5.4 Fase 4: Multimodal + Guardrails

La Fase 4 anade capacidades de vision (analisis de imagenes, capturas de pantalla) y guardrails (limites de seguridad para prevenir comportamientos peligrosos o indeseados). La vision se implementa usando modelos multimodales de Ollama como `llava` o `qwen2.5-vl`, que pueden procesar imagenes junto con texto. Los guardrails incluyen: validacion de salida (evitar que el agente genere codigo peligroso), limites de accion (maximo de comandos ejecutados por sesion), y filtros de contenido (detectar instrucciones de inyeccion de prompt). Esta fase requiere la estructura modular para poder agregar un modulo `agent/guardrails.py` sin modificar el motor ReAct.

5.5 Fase 5: Multi-Agente + Workflow Engine

La Fase 5 es la mas ambiciosa: transforma el agente unico en un sistema multi-agente donde diferentes agentes especializados colaboran en tareas complejas. Un workflow engine orquesta la comunicacion entre agentes, asigna subtareas, y fusiona resultados. Esta fase se beneficia enormemente de la refactorizacion modular, ya que cada agente es esencialmente una instancia del ReactAgent con su propio system prompt, herramientas y memoria. Sin la separacion de concerns, implementar multi-agente en un archivo de 2400+ lineas seria practicamente imposible de mantener. El workflow engine se implementa como un modulo independiente `agent/workflow.py` que coordina agentes a traves de una cola de mensajes compartida.

6. Resumen de Impacto Estimado

Optimizacion	Antes	Despues	Mejora
Busqueda vectorial (1000 entradas)	50ms	3ms	15x
Carga de VectorStore desde disco	200ms	4ms	50x
RAM de vectores (1000 entries)	25MB	8MB	3x
Cache miss rate en embeddings	40%	15%	2.7x
Tokens por llamada LLM (conv. larga)	4000	2500	1.6x
Tiempo para agregar herramienta nueva	3 archivos	1 decorador	3x
Tests unitarios posibles	0	Por modulo	Infinito
Preparacion para MCP (Fase 3)	8 puntos de edicion	1 modulo nuevo	8x

Tabla 4: Resumen de impacto cuantitativo de las optimizaciones propuestas

Recomendacion principal: Completar las optimizaciones de Fase 2 (5-8 horas) ANTES de iniciar la refactorizacion modular. Esto permite validar las mejoras de rendimiento en la arquitectura actual y tener una baseline clara antes de reestructurar el codigo. La refactorizacion modular (Fase 3) es el prerrequisito critico para las Fases 4 y 5.